

ContextLayer

.dev

The missing context layer for AI coding agents.

Make Claude Code, Cursor, and every AI agent on your codebase as smart as a senior engineer who joined yesterday and read everything — all your PRs, decisions, and team conventions.

IDEA OVERVIEW · v1

Antrang Sharma · 2026-05-18

State of Oregon Claude Code Hackathon (online, 48h)

01 · THE IDEA

What is ContextLayer.dev?

ContextLayer.dev reads your codebase's history — every commit, every pull-request discussion, every team decision — and turns it into structured knowledge that AI coding agents can search. When you ask Claude Code (or Cursor, or any other AI agent) to write code in your repo, it first looks up the relevant team conventions, past decisions, and deprecated patterns. Then it writes code that actually fits your codebase, not generic code.

"Your AI assistant becomes a senior engineer who joined yesterday and read every PR, every decision doc, and every Slack thread."

In three sentences:

- **1.** Point ContextLayer at any Git repository.
- **2.** It uses three Claude models working together to extract team conventions, design decisions, deprecated paths, and anti-patterns.
- **3.** It serves this knowledge to any AI coding agent through a standard protocol (MCP), so your AI gives codebase-aware answers from minute one.

02 · THE PROBLEM

Why this needs to exist

Every codebase has two layers of knowledge:

- **The code itself** — files, functions, types. Everyone can read this.
- **The hidden layer** — why-this-not-that decisions, team conventions, deprecated paths, lessons from incidents. This lives in PR comments, commit messages, Slack threads, and the heads of senior engineers.

AI coding agents only see the first layer. Every Claude Code session starts fresh, with zero memory of your team's conventions. So they rediscover the same context badly, every time.

A concrete example

You ask Claude Code: *"Add a new endpoint that fetches a user's billing history."*

- **Without ContextLayer:** Claude writes a plausible endpoint using exceptions for error handling, calling the legacy `billing_helper.py` module. The code works locally but doesn't fit your codebase.
- **With ContextLayer:** Claude queries the knowledge layer first, learns that your team adopted `Result` types in PR #421 (because exceptions broke async tracing), and that `billing_helper.py` was deprecated in PR #487 in favor of the new `billing/` module. It writes the correct version on the first try, with citations.

"This is the #1 reason AI coding tools produce mediocre output on real codebases. They write code that looks right but ignores the conventions you spent years building."

03 · HOW IT WORKS

From repo to context — four simple steps

ContextLayer is two pieces of software working together: an **indexer** that runs once when you set it up, and an **MCP server** that runs while you're coding.

Step 1 · Ingest

The indexer reads every commit message, every merged PR description, and every PR review comment from your repo using the standard `git` and `gh` command-line tools. No special access needed beyond what you already use.

Step 2 · Filter (Haiku)

Claude Haiku — the fast, cheap model — looks at every raw event (typically thousands) and asks a simple question: *"Does this contain a team convention, decision, deprecation, or anti-pattern?"* It throws out the 90%% that doesn't.

Step 3 · Extract (Sonnet)

Claude Sonnet — the smart middle model — takes the surviving events and extracts structured "knowledge atoms": a summary, the rationale, the scope it applies to, and links back to the source PRs. We use Anthropic's tool-use feature so the JSON is always valid.

Step 4 · Structure (Opus)

Claude Opus — the most capable model, with extended thinking enabled — reads all the extracted atoms in one pass. It removes duplicates, resolves conflicts, groups atoms into named topics, and promotes the highest-confidence ones to always-on "rules."

The output sits in a small local database. The MCP server reads from this database and exposes two simple search tools to any AI agent that speaks the Model Context Protocol (MCP) — which now includes Claude Code, Cursor, Cline, and many others.

04 · ARCHITECTURE

How we're building it

The whole system is two Python processes and one local file. No backend servers, no cloud infrastructure, no auth system. This is a deliberate constraint — it keeps the cost at **\$0** through the entire pre-investor phase, and it makes the demo bulletproof.

INDEXER (one-shot)

```
$ uvx contextlayer index .
```

Git history + PR comments → Haiku filter → Sonnet extract (tool use) → Opus structure (extended thinking)

↓ writes ↓

~/.contextlayer/<repo>/index.db (SQLite + fastembed vectors)

↑ reads ↑

MCP SERVER (long-running)

```
$ uvx contextlayer mcp --repo .
```

Exposes context_query() and context_list_topics() over stdio to any MCP client.

The technology stack

Layer	Choice	Why
Language	Python 3.11+	Where Anthropic SDK + MCP SDK live
AI calls	Claude Agent SDK	Battle-tested, handles tool use cleanly
AI models	Haiku · Sonnet · Opus	Right cost-quality fit per stage
MCP server	Official Python MCP SDK	Anthropic-maintained, ~30-line scaffold
Storage	SQLite (WAL mode)	Zero infra, one file, durable, fast
Embeddings	fastembed (ONNX, local)	~100MB install, no GPU, no API cost
Vector search	NumPy brute-force cosine	Microseconds at our scale, zero deps

Layer	Choice	Why
Distribution	uvx / PyPI	One-line install: uvx contextlayer
Landing page	Static HTML + Tailwind + Vercel	Free, fast, CI-deployed from GitHub

05 · WHO BENEFITS

Three users, three different wins

Solo developers

Anyone with an open-source repo who wants AI assistants to respect the project's conventions — without manually maintaining a CLAUDE.md or AGENTS.md by hand.

Engineering teams (5–200)

Startups where 'how we do things' lives in senior engineers' heads. ContextLayer turns that tribal knowledge into something every dev and every AI tool can query in real time.

Enterprises (200+)

Companies running thousands of devs on Copilot, Cursor, and Claude Code. Different AI tools give inconsistent answers because each rediscovers conventions independently. ContextLayer is the shared context bus.

AI agent platforms

Cursor, Cline, Aider, and custom internal agents all benefit from the same indexed knowledge. ContextLayer is the layer below them — the picks-and-shovels for the AI coding gold rush.

Use cases

- **Onboarding** — new engineers ask Claude Code about the codebase and get senior-engineer answers from day one, with citations.
- **Code review** — AI reviewers can flag PRs that violate established conventions, with links to the original decision.
- **Senior engineer leverage** — instead of answering the same questions in Slack for the tenth time, conventions live in a queryable layer.
- **Cross-tool consistency** — Cursor, Claude Code, and Copilot all give the same answer because they all query the same knowledge.
- **Knowledge preservation** — when senior engineers leave, their context stays behind in a queryable form.

06 · BENEFITS AT A GLANCE

What you actually get

<\$1 Cost to index a repo	3–7 Minutes to first index	\$0 Infra cost (BYOK)	Any AI agent that speaks MCP
--	--------------------------------------	---------------------------------	---

Codebase-aware answers from minute one

No more priming Claude Code with copy-pasted context every session. The knowledge is already there, queryable, scoped to the files you're editing.

Works with every AI agent you already use

MCP is the protocol Claude Code, Cursor, Cline, and many others speak. One index serves all of them.

Stays current with one command

Re-run `contextlayer index` . after a sprint and the knowledge layer updates. The idempotent cache means re-indexing is fast and cheap.

Free forever for individuals (BYOK)

Bring your own Anthropic API key. The open-source tool is free. You pay only for the Claude calls you make — typically under \$1 per repo.

Built on standards

Model Context Protocol is becoming the standard for tool-aware AI. ContextLayer is a first-class MCP citizen — no proprietary lock-in.

Demo-friendly, judge-friendly, investor-friendly

Before/after demos are dramatic and obvious. Anyone who's used Claude Code on a real codebase understands the pain immediately.

07 · THE COMPETITIVE EDGE

Is there anything like this?

Several tools touch adjacent problems. None of them sit in this exact slot.

Product	What it does	What it misses
GitHub Copilot Spaces	Enterprise knowledge attached to Copilot	Enterprise-only, manual setup, shallow extraction
Sourcegraph Cody	Code-graph aware AI assistant	Knows the code, not the conventions or decisions
Glean	Enterprise knowledge search	Built for docs and Slack, not code-specific
CursorRules / .cursorrules	Static rules file Cursor reads	Manual, doesn't scale, Cursor-only
CLAUDE.md / AGENTS.md	Hand-written instructions for AI agents	Manual, gets stale, doesn't surface PR rationale
ContextLayer.dev	Auto-extracted knowledge atoms served via MCP	Sits exactly in this slot — no incumbent

"The competitive moat isn't "we built it first." It's "we built the layer everyone else will need to integrate with — and we open-sourced it so the integration is theirs to do."

08 · WHY IT WILL WORK

Five reasons this is the right window

1 · Every company is shipping AI coding agents in 2026

The market doesn't need to be created — it's already racing. Every team is hitting the same wall: agents that don't respect codebase conventions. ContextLayer sells the picks-and-shovels.

2 · Anthropic is making MCP the standard

MCP shipped in late 2024 and is now in Claude Code, Cursor, Cline, and dozens of other clients. Being MCP-native means ContextLayer rides Anthropic's distribution — every new MCP-capable client is a new compatible host, for free.

3 · No incumbent in this exact slot

Cursor is the IDE. Claude Code is the harness. Copilot is the assistant. None of them are the cross-cutting context layer. The slot is open.

4 · Open-source distribution = free bottom-up GTM

Dev tools win by getting one developer at a company to fall in love. Linear, Cursor, Vercel, Notion — all did this. Open-source the core, let devs install for free, sell hosted + SSO to the company that already loves it.

5 · The acquisition story is real

Anthropic, GitHub, Cursor, Vercel, and JetBrains each have a strategic reason to own this layer. Comparable agent-tooling exits are in the \$50M–\$500M range at \$5M+ ARR. The path from hackathon to acquisition-ready is plausible in 12–18 months.

09 · BUILD PLAN

Hackathon to startup in five phases

Phase	When	What ships
1 · Hackathon MVP	May 18–20, 2026 (48h)	Working CLI + MCP server, FastAPI demo, landing pag
2 · OSS launch	June 2026	HN launch, PyPI release, polish docs, growth to ~500 G
3 · Hosted SaaS tier	Q3 2026	Managed indexing, multi-repo, Slack + Linear adapters;
4 · Team + Enterprise	Q4 2026	Team-wide knowledge sharing, SSO, audit logs, on-pre
5 · Scale or exit	2027+	Hit \$5M+ ARR; either raise to scale or accept acquisition

48-hour hackathon sequence

The discipline that makes the hackathon ship: **build minimum viable demo end-to-end first, then polish.**
Not pipeline → MCP → demo linearly.

- **Hours 0–6** · Project setup, ingestion adapters, first Claude API call working.
- **Hours 6–18** · Multi-agent pipeline end-to-end on a tiny synthetic repo. Atoms in SQLite.
- **Hours 18–24** · MCP server scaffolded with Python SDK. Claude Code can query atoms.
- **Hours 24–32** · Build polished synthetic 15-PR demo repo. Lock the demo question.
- **Hours 32–40** · Pre-index FastAPI as the optional "wow" demo. Landing page. Slide deck.
- **Hours 40–48** · Record demo video, edit, rehearse Q&A, submit.

10 · BUSINESS MODEL

How it makes money

Standard open-core SaaS playbook. Free OSS for individuals, paid tiers for teams and enterprise. Same pattern as Cursor, Linear, Vercel.

Tier	Price	What's included	Comparable
Free / OSS	\$0 + BYOK	Self-hosted, MCP server, all ingestion adapters	
Pro	\$20 / seat / mo	Hosted indexing, multi-repo, Claude + Pin	Cursor + Pin \$20
Team	\$50 / seat / mo	Team knowledge sharing, additional MCPs	OpenAI \$10-\$100s
Enterprise	\$50K–500K / yr	SSO, on-prem, custom ingestion, SSO, SSO	Sourcegraph

Go-to-market (first 90 days post-launch)

- **Show HN launch** — "The missing context layer for Claude Code." Dev-tools audience, MCP novelty, dramatic demo = front-page material.
- **Anthropic Discord + Claude Code community** — drop-in MCP, frictionless install path, immediate value.
- **Bottom-up enterprise** — devs adopt free tier, team upgrades to hosted, CTO signs the SSO contract. Same path Linear and Cursor walked.

11 · HONEST RISKS

What could go wrong (and what we do about it)

Risk	Likelihood	Mitigation
Anthropic builds this natively into Claude Code	Medium	If they do, they probably buy the leader instead. Be the leader.
Cursor adds it natively to their IDE	Medium	Cursor's incentive is the IDE, not the cross-cutting layer.
Atom extraction quality varies by codebase	High (early)	Multi-agent pipeline + Opus pass mitigates. We tune prompts.
MCP ecosystem stalls	Low	Already shipped in 5+ major clients in <12 months. Momentum.
Slow open-source adoption	Medium	HN launch, dev influencer outreach, free tier removes friction.
Demo doesn't land on stage	Low (with mitigation)	Two pre-warmed Claude Code panes, deterministic synthesis.

"The most honest thing about this idea: even the worst-case outcome — that Anthropic or GitHub builds it themselves — is a buying signal, not a losing scenario."

12 · ONE-PAGE SUMMARY

Everything in a glance

The bet

Every AI coding agent rediscovers codebase context badly, every session. ContextLayer extracts that context once and serves it to every agent. It's the missing infrastructure layer in the 2026 AI coding stack.

The product

- Two Python processes (indexer + MCP server), one local SQLite file.
- Multi-agent pipeline: Haiku filters, Sonnet extracts with tool use, Opus structures with extended thinking.
- Standard MCP interface — any compatible AI agent can query the knowledge.
- Open-source core (MIT), bring-your-own-API-key. \$0 infra cost.

The market

- **Individuals:** free OSS, BYOK.
- **Teams:** \$20–50 / seat / month for hosted indexing + integrations.
- **Enterprise:** \$50K–500K / year for SSO, on-prem, audit.

The arc

- **Now:** 48-hour hackathon demo, public GitHub repo, landing page.
- **3 months:** HN launch, 500+ stars, first paying teams.
- **12 months:** \$1–2M ARR, 5+ enterprise pilots.
- **18 months:** \$5M+ ARR, acquisition-eligible.

Repo · github.com/AntarangSharma/ContextLayer.dev

Spec · [docs/specs/2026-05-18-contextlayer-design.md](https://docs.specs/2026-05-18-contextlayer-design.md)

Hackathon · State of Oregon Claude Code Hackathon — demo Wed 2026-05-20